

Phase 1 Baseline Limit Order Book in Modern C++20

End-to-End Deep-Dive: Development Toolchains, Domain Types, FIFO Matching Loop, Unit Tests, and RDTSC Latency Profiling

Target Stack: Modern C++20 / MSVC 19.51 Optimizing Compiler	Author / Engineer: Akash
Project Phase: Phase 1 Baseline (FIFO Price-Time Priority)	Deliverable: Verified Core + RDTSC Hardware Benchmark

Week 1 Mission: The goal of Week 1 was *not* premature micro-optimization. The mission was to establish an indisputable, 100% semantically correct baseline matching engine using standard C++ STL data structures (`std::map` and `std::list`), verify all exchange matching invariants via automated unit tests, and profile execution latency using CPU hardware clock cycles (`__rdtsc()`) to discover the empirical memory bottlenecks that justify Phase 2.

1. Toolchain & Environment Configuration

Low-latency systems require native 64-bit compilation and modern language features. Here is how the environment was set up:

Component	Discovered / Configured	Low-Latency Architectural Impact
Compiler Discovery	MSVC 19.51 (Visual Studio 2026 Developer BuildTools) detected.	Replaced legacy MinGW 6.3.0 (2016) with an enterprise-grade C++20 optimizing compiler.
Environment Init	vcvars64.bat native 64-bit shell initialization.	Configures 64-bit pointer architecture, access to <code>cl.exe</code> , and x86 hardware intrinsics.
Optimization Flags	<code>/std:c++20 /O2 /EHsc /W4</code>	Enforces strict C++20 ISO standards, aggressive inlining, loop unrolling, and maximum warning scrutiny.
Automation Script	build.bat one-click compilation pipeline.	Automatically builds unit tests, executes test suite, builds benchmark harness, and runs 100k-order test.
Version Control	Git repository initialized with <code>.gitignore</code> .	Initial commit recorded: establishes chronological engineering audit trail for interviews.

2. Domain Model & Foundation Types

Low-latency architecture begins with careful representation of domain data. Let's inspect `Types.hpp` and `Order.hpp`:

File: `src/common/Types.hpp`

- **enum class Side : uint8_t { BUY = 0, SELL = 1 };**
 - *Why enum class?* Strongly typed; prevents accidental comparisons against raw integers.
 - *Why : uint8_t?* Default C++ enums take 4 bytes. Specifying `uint8_t` packs it into **1 single byte**, minimizing cache-line footprint.
- **using Price = uint64_t; (Fixed-Point Arithmetic):**
 - *The Floating-Point Trap:* In IEEE 754 floating-point, $0.1 + 0.2 \neq 0.3$ due to binary fraction rounding noise. Exchanges cannot tolerate price calculation drift. We store prices as 64-bit integer ticks (e.g. \$100.25 is stored as 10025 cents). Comparisons take a single CPU cycle with zero floating-point unit (FPU) stalls.
- **using Quantity = uint32_t;** Represents open shares or contracts up to 4.29 billion.
- **using OrderId = uint64_t;** Monotonically increasing unique 64-bit identifier.
- **using Timestamp = uint64_t;** Nanosecond timestamp or hardware CPU cycle counter recorded at ingress.
- **struct Trade:** Emitted whenever an incoming order crosses resting liquidity:

```
struct Trade {
    OrderId maker_order_id; // Resting passive order already in the book
    OrderId taker_order_id; // Aggressive incoming order that took liquidity
    Price price; // Execution price (ALWAYS set by maker's resting price)
    Quantity quantity; // Number of shares executed
    Timestamp timestamp; // Execution timestamp
};
```

File: `src/common/Order.hpp`

Represents an individual order instance in the system:

- `quantity` vs. `initial_quantity`: As partial fills occur, `quantity` decrements. Retaining `initial_quantity` allows microstructure estimators to track fill ratios.
- `is_filled()` `const`: Inline helper returning `quantity == 0`.
- `print()` `const`: Formats order details for console debugging.

3. Core Matching Engine Architecture & Walkthrough

In `OrderBookBaseline.hpp` and `OrderBookBaseline.cpp`, the limit order book is organized using textbook STL containers:

Data Structures Under the Hood:

- `std::map<Price, std::list<Order>, std::greater<Price>> bids_;` A Red-Black tree storing Bids sorted in descending order. `bids_.begin()` always points to the **Best Bid** (highest price).
- `std::map<Price, std::list<Order>, std::less<Price>> asks_;` A Red-Black tree storing Asks sorted in ascending order. `asks_.begin()` always points to the **Best Ask** (lowest price).
- `std::list<Order>`: A doubly-linked list representing the queue of resting orders at a specific price level in FIFO time order.
- `std::unordered_map<OrderId, std::pair<Side, Price>> order_index_;` Hash map enabling fast $O(1)$ lookup for cancellations.

The 6-Step Matching Algorithm (Inside `match_buy_order`):

Step 1: Spread Verification: While incoming buy quantity > 0 and asks exist, inspect `best_ask = asks_.begin()`. If `incoming_buy.price < best_ask_price`, the spread is not crossed. Matching halts.

Step 2: FIFO Queue Traversal: Access the resting order queue at `best_ask_price`. Inspect the oldest order at `front()`.

Step 3: Trade Execution: Matched quantity is $\min(\text{incoming_buy.qty}, \text{resting_ask.qty})$. A Trade event is emitted at the maker's resting price.

Step 4: Quantity Decrement: Subtract `fill_qty` from both incoming buy and resting ask. If the resting ask hits 0, pop it from the queue and remove from `order_index_`.

Step 5: Empty Level Pruning: If the price level queue becomes empty, erase the price node from `asks_`.

Step 6: Passive Resting: If incoming buy order still has open shares, append it to `bids_[price].push_back()` and register in `order_index_`.

Order Cancellation Routine (`cancel_order`):

1. Look up `order_id` in `order_index_`. If absent, return false.
2. Retrieve the order's Side and Price; find the corresponding price level list in `bids_` or `asks_`.
3. Traverse the list, erase the matching order, clean up the price level if empty, and return true.

4. Correctness Test Suite (BaselineTests.cpp)

Financial systems require zero tolerance for execution bugs. BaselineTests.cpp validates 6 mission-critical exchange invariants:

Unit Test Function	Scenario Tested	Expected Invariant Verified
test_passive_resting	Buy @ \$100.00, Sell @ \$100.50 submitted.	Zero trades generated; BBO correctly reflects \$100.00 Bid and \$100.50 Ask.
test_exact_fill	Resting Ask 100 @ \$100; Incoming Buy 100 @ \$100.	1 Trade emitted for 100 shares; book is left completely empty.
test_partial_fill	Resting Ask 200 @ \$100; Incoming Buy 50 @ \$100.	1 Trade for 50 shares; 150 shares remain resting on the Ask level.
test_multi_level_price_sweep	Asks at \$100.00, \$100.05, \$100.10. Buy 200 @ \$100.10.	Sweeps Level 1 and 2 completely, partially fills Level 3; 3 distinct trades emitted.
test_fifo_time_priority	Order A arrives before Order B at the same price.	Incoming cross fills Order A completely before Order B receives a single share.
test_order_cancellation	Cancel resting order; re-cancel same order.	First cancel succeeds and decrements depth; second cancel returns false.

Console Order Book Depth Ladder Output:

```
===== ORDER BOOK DEPTH =====
ASKS (SELLERS - Lowest First)
-----
Price ($) Shares Orders
150.15 5000 1
150.10 2500 1
150.05 800 1
-----
>>> SPREAD: $0.05 <<<
-----
BIDS (BUYERS - Highest First)
-----
150.00 500 1
149.95 1200 1
149.90 3000 1
=====
```

5. Hardware Benchmarking & The Profiling Epiphany

Using benchmark/BaselineBenchmark.cpp, we calibrated the CPU hardware clock frequency and timed 100,000 synthetic orders:

Metric	Measured Value	Systems Analysis / Low-Latency Interpretation
Orders Processed	100,000 orders	50% Buys, 50% Sells, randomized prices around \$100.00.
Trades Executed	78,243 trades	Demonstrates active matching across dynamic bid/ask spreads.
Total Wall Time	21 milliseconds	Whole test completed in a fraction of a blink.
Throughput	4,761,905 orders/sec	Warm cache single-threaded execution rate.
p50 Latency (Median)	149.8 nanoseconds	Typical fast-path insertion when cache lines are hot.
p90 Latency	297.7 nanoseconds	Slight degradation under multi-level traversals.
p99 Latency	585.7 nanoseconds	Beginning of Red-Black tree pointer chasing overhead.
p99.9 Latency	2,740.8 nanoseconds (2.7µs)	18x degradation from median due to memory allocator stalls.
Max Latency (Outlier)	273,349.9 ns (273.3µs)	1,824x LATENCY SPIKE! Caused by heap allocator locks and page faults.

The Phase 2 Mandate: Why This Number Wins Interviews:
Notice that while the median latency is **149.8 nanoseconds**, the worst-case outlier spikes to **273 microseconds** (a 1,824x degradation). In a real electronic trading environment, a 273-microsecond pause during high volatility results in severe slippage and adverse selection. This empirical data provides the exact justification for **Phase 2 (Weeks 2-3)**: replacing std::map and std::list with a **Contiguous Slab Allocator (Object Pool)** and **Flat Direct-Indexed Price Ladder**, eliminating all dynamic heap allocations and crushing tail latency.